

Payment System API Documentation

This document explains all the API endpoints in the payment system, what inputs they need, and what they return.

Authentication

Most endpoints require authentication using a Bearer token in the header:

```
Authorization: Bearer your-token-here
```

Main Payment Endpoints

1. Create Checkout Session

Endpoint: `POST /create-checkout-session`

What it does: Creates a Stripe payment session for a user to buy a subscription

Requires Authentication: Yes

Input (JSON Body):

```
json
{
  "price_id": "price_1PG...monthly"
}
```

Input Fields:

- `price_id` (required): The Stripe price ID for the subscription plan (monthly or annual)

Response:

```
json
{
  "checkout_url": "https://checkout.stripe.com/...",
  "session_id": "cs_test_..."
}
```

What happens:

- Creates a Stripe customer if the user doesn't have one
- Creates a checkout session that expires in 59 minutes

- Returns a URL where the user can complete payment
-

2. Stripe Webhook

Endpoint: `POST /webhook`

What it does: Receives notifications from Stripe when payments are completed

Requires Authentication: No (secured by Stripe signature)

Input: Raw webhook payload from Stripe

What happens:

- Verifies the webhook is really from Stripe
 - When a payment is completed, upgrades the user to Premium
 - Records the transaction in the database
 - Prevents duplicate processing
-

3. Check Session Status

Endpoint: `GET /session-status/{session_id}`

What it does: Checks the status of a payment session

Requires Authentication: Yes

Input (URL Parameter):

- `session_id`: The checkout session ID (like "cs_test_...")

Response:

```
json
{
  "status": "complete",
  "payment_status": "paid"
}
```

Subscription Status Endpoints

4. Get Subscription Status by User ID

Endpoint: `GET /subscription-status/{user_id}`

What it does: Gets subscription details for a specific user

Requires Authentication: No

Input (URL Parameter):

- `user_id`: The user's UUID

Response:

```
json
{
  "user_id": "123e4567-e89b-12d3-...",
  "subscription_status": "Premium",
  "is_active": true,
  "expires_at": "2025-09-08",
  "days_remaining": 30,
  "current_plan": "Monthly",
  "payment_amount": 29.99,
  "payment_date": "2025-08-09"
}
```

5. Get Subscription Status by Email

Endpoint: `GET /subscription-status/by-email/{email}`

What it does: Same as above, but uses email instead of user ID

Requires Authentication: No

Input (URL Parameter):

- `email`: The user's email address

Response: Same as above, plus the email field

Admin/Cron Endpoints

6. Update Expired Subscriptions (Cron Job)

Endpoint: `POST /tasks/update-expired-subscriptions`

What it does: Automatically downgrades expired Premium users to Free

Requires Authentication: Yes (special cron token)

Input: None

Response:

```
json

{
  "status": "success",
  "updated_count": 5
}
```

Testing Endpoints

7. Create Test User

Endpoint: `POST /test/create-user`

What it does: Creates a test user for development/testing

Requires Authentication: No

Input (JSON Body):

```
json

{
  "email": "test@example.com",
  "name": "Test User",
  "subscription_status": "Free trial"
}
```

Input Fields:

- `email` (required): User's email
- `name` (required): User's name
- `subscription_status` (optional): Default is "Free trial"

8. Manual Upgrade User

Endpoint: `POST /test/manual-upgrade`

What it does: Manually upgrades a user to Premium for testing

Requires Authentication: No

Input (JSON Body):

```
json
```

```
{
  "email": "test@example.com",
  "plan_type": "monthly",
  "test_mode": true
}
```

Input Fields:

- `email` (required): User's email
- `plan_type` (required): Either "monthly" or "annual"
- `test_mode` (optional): Default is true

Response:

```
json
{
  "success": true,
  "message": "✅ Successfully upgraded test@example.com to Monthly Test",
  "user_id": "123e4567-e89b-12d3-...",
  "expires_at": "2025-09-08",
  "plan_details": {
    "plan_name": "Monthly Test",
    "duration_days": 30,
    "payment_amount": 29.99
  },
  "test_session_id": "test_session_abc123"
}
```

9. Get Test User Status

Endpoint: `GET /test/user-status/{email}`

What it does: Gets detailed user info for testing

Requires Authentication: No

Input (URL Parameter):

- `email`: User's email address

Response: Detailed user information including subscription status

10. Reset Test User

Endpoint: `DELETE /test/reset-user/{email}`

What it does: Resets a user back to free trial status

Requires Authentication: No

Input (URL Parameter):

- `email`: User's email address

Response:

```
json

{
  "success": true,
  "message": "✅ Reset test@example.com to free trial status",
  "updated_records": 1
}
```

11. Get User Transactions

Endpoint: `GET /test/transactions/{email}`

What it does: Shows all payment transactions for a user

Requires Authentication: No

Input (URL Parameter):

- `email`: User's email address

Response:

```
json
```

```
{
  "found": true,
  "user_id": "123e4567-e89b-12d3-...",
  "email": "test@example.com",
  "transaction_count": 2,
  "transactions": [
    {
      "stripe_session_id": "cs_test...",
      "user_id": "123e4567-e89b-12d3-...",
      "payment_amount": 29.99,
      "processed_at": "2025-08-09T10:30:00"
    }
  ]
}
```

12. System Status

Endpoint: `GET /test/system-status`

What it does: Shows overall system health and statistics

Requires Authentication: No

Input: None

Response:

json

```
{
  "system_status": "✅ Operational",
  "database_connected": true,
  "environment_variables": {
    "SUPABASE_URL": "✅ Set",
    "STRIPE_SECRET_KEY": "✅ Set"
  },
  "user_statistics": {
    "total_users": 150,
    "status_breakdown": {
      "Free trial": 120,
      "Premium": 30
    }
  },
  "transaction_statistics": {
    "total_transactions": 45
  }
}
```

13. Health Check

Endpoint: `GET /health`

What it does: Simple health check for the payment system

Requires Authentication: No

Input: None

Response:

```
json
{
  "status": "healthy",
  "timestamp": "2025-08-09T10:30:00.000Z",
  "services": {
    "database": "✅ Connected",
    "stripe": "✅ Configured",
    "environment": "✅ All variables set"
  }
}
```

Common Response Fields

Subscription Status Values:

- `"Free trial"` - User is on free trial
- `"Premium"` - User has active paid subscription
- `"Free"` - User's subscription expired

Plan Types:

- `"monthly"` - 30 day subscription
- `"annual"` - 365 day subscription

Payment Status Values:

- `"paid"` - Payment completed successfully
 - `"unpaid"` - Payment not completed
 - `"no_payment_required"` - No payment needed
-

Error Responses

All endpoints return error responses in this format:

```
json

{
  "detail": "Error message explaining what went wrong"
}
```

Common HTTP status codes:

- `400` - Bad request (invalid input)
 - `401` - Unauthorized (missing/invalid token)
 - `403` - Forbidden (permission denied)
 - `404` - Not found (user/session doesn't exist)
 - `500` - Internal server error
 - `502` - External service error (Stripe)
 - `503` - Service unavailable (database down)
-

Required Environment Variables

The system needs these environment variables to work:

- `STRIPE_SECRET_KEY` - Your Stripe secret key

- `STRIPE_WEBHOOK_SECRET` - Stripe webhook signing secret
 - `SUPABASE_URL` - Your Supabase database URL
 - `SUPABASE_KEY` - Your Supabase API key
 - `PAYMENT_SUCCESS_URL` - Where to send users after successful payment
 - `PAYMENT_CANCEL_URL` - Where to send users if they cancel payment
 - `CRON_SECRET_KEY` - Secret key for cron job authentication
-

Testing Flow

1. **Create a test user:** `POST /test/create-user`
 2. **Check their status:** `GET /test/user-status/{email}`
 3. **Upgrade them manually:** `POST /test/manual-upgrade`
 4. **Verify the upgrade:** `GET /test/user-status/{email}`
 5. **Check transactions:** `GET /test/transactions/{email}`
 6. **Reset when done:** `DELETE /test/reset-user/{email}`
-

Production Flow

1. **User wants to subscribe:** Frontend calls `POST /create-checkout-session`
2. **User pays:** Redirect to `checkout_url` from response
3. **Payment completes:** Stripe calls `POST /webhook` automatically
4. **Check status:** Frontend can call `GET /session-status/{session_id}`
5. **View subscription:** Use `GET /subscription-status/by-email/{email}`